VERIQTA

CLOUD SECURITY REVIEW HANDBOOK

A Production Guide to Auditing, Securing and Approving Cloud Environments



VERIQTA

COVERS

- ✓ Cloud Architecture Review
- ✓ IAM and Trust Relationships
- ✓ Kubernetes and Workload Security
- ✓ Data Protection and Recovery
- ✓ CI/CD and Supply Chain Security
- ✓ Detection and Forensics
- ✓ Real-World Attack Paths
- ✓ Risk Prioritization
- ✓ Security Decision Framework
- ✓ Production Approval Review

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

01

THE ARCHITECTURE DIAGRAM IS LYING

Why Your Documented Design Can Look Secure While the Actual Environment Is Exposed

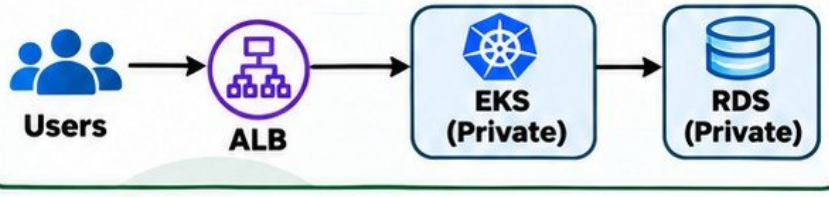
THE PROBLEM

The documented architecture often shows a clean, isolated system, but the deployed environment can have hidden trust relationships, extra access paths, and unmanaged connections that create real risk.

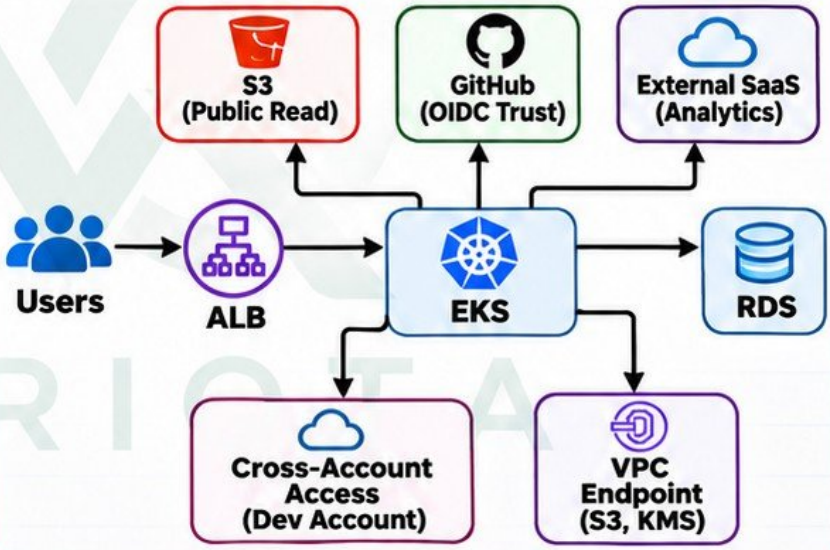
WHERE THE RISK HIDES

- **Internet exposure** (via load balancers, storage, or misconfigured services)
- **Private endpoints** (with overly broad access)
- **Cross-account routes** (e.g. VPC peering, Transit Gateway)
- **Workload identities** (roles with more access than intended)
- **Unmanaged dependencies** (e.g. third-party SaaS, registries, analytics, backups)

DOCUMENTED (EXPECTED)



ACTUAL (DEPLOYED)



REVIEW APPROACH

1. Compare the design vs. real resources.
2. Map all trust relationships (roles, policies, federations).
3. Trace network paths (internet, private endpoints, peering, route tables).
4. List external and third-party dependencies.
5. Validate with real evidence (console, CLI, logs).

SENIOR ENGINEER INSIGHT

A system can have **no publicly accessible servers** and still expose sensitive data through trusted services and indirect access paths.

VERIQTA

02

THE PERMISSIONS NOBODY ASSIGNED

How Effective Permissions Emerge from Role Chains, Resource Policies, Organizations and Trust

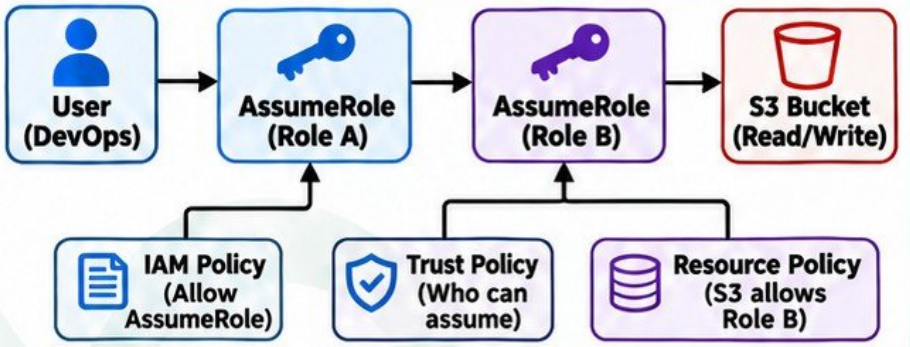
THE IDEA

An identity's real permissions are not just what you see in one IAM policy. They come frim a combination of policies, trust relationships, and service behaviors across your environment.

KEY SOURCES OF EXTRA PERMISSIONS

- ✓ IAM Role Chains (AssumeRole to gain more privileges)
- ✓ Resource Policies (e.g. S3, KMS, Lambda allowing access)
- ✓ Organization Boundaries (SCPs, delegated admin, account inheritance)
- ✓ Session Policies (temporary restrictions or expansions)
- ✓ Service-Linked Roles (roles AWS creates with broad service acccss)

EXAMPLE: HOW PERMISSIONS STACK



EFFECTIVE PERMISSION FLOW

Identity (User) + Role Chain + Resource Policy + Org / SCP Context → Session Policy = Effective Permissions

COMMONLY MISSED SCENARIOS	WHAT CAN HAPPEN
Role chaining across accounts	Higher privileges than expected
Review resource policies (S3, KMS, etc.).	Access even if your IAM policy doesn't allow it
Service-linked role (e.g. ECS, EKS)	Broad service access to create or modify resources
Org / SCP misconfiguration	Permissions remain allowed in other accounts
Session policy not reviewed	Temporary session gains more access than intended

REVIEW CHECKLIST

- ✓ Trace all role assumptions (role chain).
- ✓ Review resource policies (S3, KMS, etc.).
- ✓ Check organization SCPs and account boundaries.
- ✓ Inspect session policies from federation (e.g. OIDC, SSO).
- ✓ Identify service-linked roles and their scope.

SENIOR ENGINEER INSIGHT

Reviewing individual IAM policies is not enough. Always evaluate the combined effect of identities, resource policies, trust, and organizational controls to understand the true access an identity has.

VERIQTA

03

PRIVATE NETWORKING, PUBLIC CONSEQUENCES

How "Private" Cloud Networks Can Still Create Real-World Exposure and Data Loss

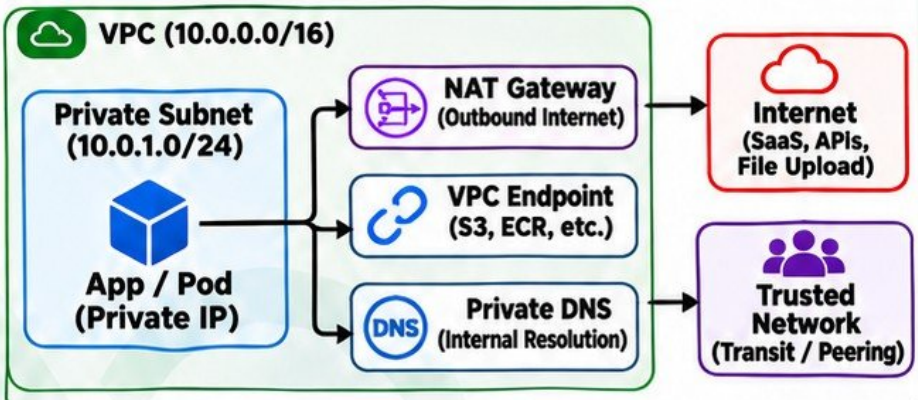
THE REALITY

A cloud environment can use private IPs, private subnets and VPC endpoints, yet still be exposed through outbound access, shared routes, misconfigurations and trusted service paths.

HIDDEN ATTACK PATHS

- NAT Gateway – allows outbound access to the internet (data exfiltration risk).
- VPC Endpoints – give access to cloud services (g. S3) that can leak data.
- Private DNS – can be poisoned or misconfigured to redirect traffic.
- Transit Routing – can extend access to other VPCs or accounts.
- Load Balancers – internal LBs can still be reachable from peered networks.
- Outbound Connections – applications can send data to external SaaS APIs.

EXAMPLE ARCHITECTURE: "PRIVATE" BUT EXPOSED



REAL-WORLD CONSEQVNCES

- Data Exfiltration
- Lateral Movement (via Transit)
- Unapproved External Access
- Bypassed Security Controls
- Exposure Through Trusted Services

REVIEW AND MITIGATION

1. Inventory all egress paths (NAT, internet, SaaS, APIs).
2. Review VPC endpoints and the data they can access.
3. Validate DNS controls and prevent unauthorized redirects.
4. Audit transit gateways and peering routes.
5. Enforce egress controls (firewall, proxy, allow-list).

SENIOR ENGINEER INSIGHT

Private IP addressing limits direct reachability. It does not establish workload trust or prevent data exfiltration.

VERIQTA

04

ONE COMPROMISED WORKLOAD, TWELVE POSSIBLE PATHS

How a Single Breach Can Expand Across Your Cloud Environment

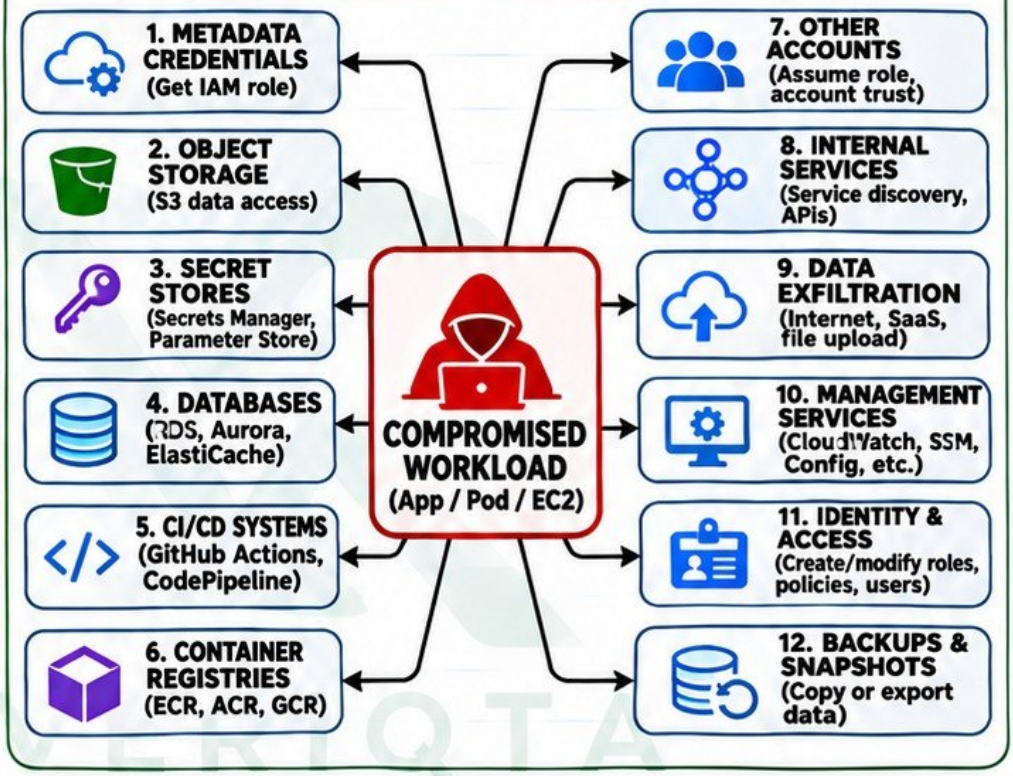
THE SCENARIO

An attacker gains control of an application workload (e.g., a container, EC2, or serverless function) and uses its identity, network access and configured permissions to move laterally across the environment.

KEY ENABLERS

- Instance/Pod metadata credentials (IAM role)
- Overly broad IAM permissions
- Accessible internal services and endpoints
- Stored secrets and configuration
- Cross-account or federated access
- Trust in CI/CD and deployment systems

THE 12 POSSIBLE MOVEMENT PATHS



INVESTIGATION FLOW



MITIGATION & HARDENING

- Use least-privilege IAM roles.
- Restrict metadata access (IMDSv2).
- Limit cross-account and federation trust.
- Segment workloads with network policies.
- Apply egress controls and DLP.
- Monitor for unusual role use and data access.

SENIOR ENGINEER INSIGHT

The severity of an initial compromise depends heavily on the privileges and reachable systems beyond the first workload.

VERIQTA

05

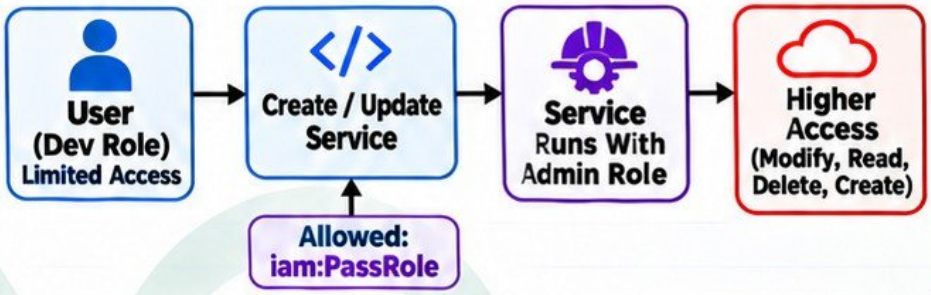
WHEN LEAST PRIVILEGE STILL ALLOWS ESCALATION

How "Limited" Permissions Can Be Chained to Gain High-Privilege Access

THE IDEA

Even with least privilege, an identity can gain higher access by using allowed actions such as **iam:PassRole**, service configuration, policy changes, and creating new workloads.

EXAMPLE: ESCALATION USING PassRole



COMMON ESCALATION VECTORS

- ✓ **iam:PassRole** – attach a high-privilege role to a service.
- ✓ **Deployment services** – use CodePipeline, ECS, EKS, or Lambda to run with elevated roles.
- ✓ **Policy modification** – update inline or resource policies to grant access.
- ✓ **Role assumption** – assume another role via trust relationships.
- ✓ **Workload creation** – launch EC2, container, or Lambda with a powerful role.

OTHER REAL-WORLD SCENARIOS

SCENARIO	HOW IT ESCALATES
PassRole Abuse	Attach an admin role to ECS, Lambda or EC2 and run commands with elevated access.
CI/CD Misuse	Use CodePipeline/Actions to deploy resources with a powerful role.
Policy Change	Modify a resource policy (e.g., S3, KMS) to grant broader access.
Role Assumption	Assume a role in another account via trust misconfiguration.
Workload Creation	Create a new EC2/EKS pod with an attached high-privilege role.

REVIEW CHECKLIST

- ✓ Identify who can use **iam:PassRole** and which roles.
- ✓ Review deployment service permissions (ECS, EKS, Lambda, CodePipeline).
- ✓ Audit IAM policy changes (cloudTrail, recent updates).
- ✓ Check role trust policies and cross-account assumptions.
- ✓ Detect new workloads with high-privilege roles.

SENIOR ENGINEER INSIGHT

An identity may lack direct administrator permissions yet still gain powerful access by configuring another service to act in its behalf.

VERIQTA

06

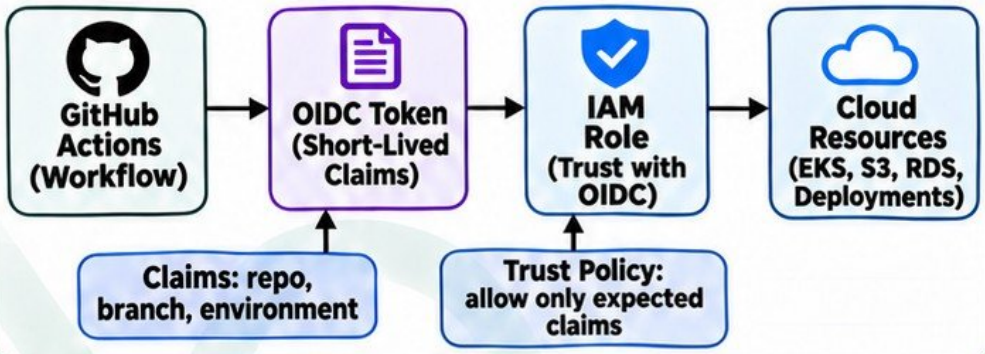
THE CI/CD PIPELINE AS A CLOUD IDENTITY BROKER

How CI/CD Uses OIDC and Federation to Get Temporary Cloud Access

THE IDEA

A CI/CD pipeline can obtain temporary, role-based access to your cloud using OIDC (OpenID Connect), without long-lived secrets, but only if the trust and claims are correctly restricted.

ARCHITECTURE: GITHUB ACTIONS → CLOUD ACCESS



KEY COMPONENTS

- ✓ **OIDC Trust** – federates GitHub Actions to your cloud IAM.
- ✓ **Claims** – conditions (e.g., repo, branch, environment).
- ✓ **Reusable Workflows** – shared pipelines across multiple repos.
- ✓ **Artifact Integrity** – ensure the build output is trusted and unchanged.
- ✓ **Deployment Role** – least-privilege role assumed by the pipeline.

CRITICAL REVIEW AREAS

AREA	WHAT TO CHECK
OIDC Trust	Identity provider, audience, and issuer are correct.
Claims	Restrict by repo, branch (e.g., main), and environment (e.g., prod).
Reusable Workflows	Ensure trusted source and avoid privilege creep.
Artifact Integrity	Use signed artifacts, checksums, and provenance (e.g., SLSA).
Deployment Role	Least privilege, scoped to only required resources.

COMMON MISCONFIGRATIONS

- ✗ Trust policy too broad (e.g., any repo or branch).
- ✗ Reusable workflow from an untrusted repository.
- ✗ Overly permissive deployment role (e.g., admin access).
- ✗ No artifact signing or integrity verification.
- ✗ Same role used for both dev and production.

SENIOR ENGINEER INSIGHT

Short-lived credentials reduce persistent secret exposure, but weak federation conditions can allow an unintended workflow to obtain privileged cloud access.

VERIQTA

07

THE SECRET WAS ROTATED. THE ATTACKER STILL HAS ACCESS.

Why Rotation Alone May Not Remove an Attacker's Access

THE IDEA

Rotating a secret (e.g., password, access key, or token) is important, but it does not always immediately remove existing access or access obtained through other trusted identities.

WHAT REALLY HAPPENS IN BREACH



KEY CONCEPTS

- ✓ **Credential Rotation** – replace an existing secret with a new one.
- ✓ **Session Revocation** – terminate active sessions and issued tokens.
- ✓ **Token Validity** – the period a token remains usable (e.g., STS, OIDC, JWT).
- ✓ **Service Access** – access gained through another trusted identity (e.g., role, federated login).
- ✓ **Identity Recovery** – contain, restore, and re-secure a compromised identity.

ROTATION VS. SESSION REVOCATION

ACTION	WHAT IT DOES	KEY LIMITATION
Rotate Secret	Creates a new credential.	Does not automatically invalidate existing sessions.
Revoke Session	Terminates active sessions/tokens.	Must be done explicitly (e.g., revoke STS/OAuth/JWT).
Token Validity	Defines how long issued tokens work.	Existing tokens remain valid until they expire.
Service Access	Access via another trusted identity (e.g., IAM role).	Rotation does not remove access from other roles or federated paths.

RECOVERY AND MITIGATION STEPS

1. Rotate all exposed credentials (keys, passwords, tokens).
2. Revoke active sessions and invalidate tokens (e.g., AWS STS, OAuth).
3. Remove or restrict alternative access paths (roles, federation, PassRole).
4. Review access logs (CloudTrail, audit logs) for continued activity.
5. Re-issue least-privilege access and monitor for re-compromise.

SENIOR ENGINEER INSIGHT

Rotating a secret does not necessarily invalidate previously issued sessions or eliminate access obtained through another trusted identity.

VERIQTA

08

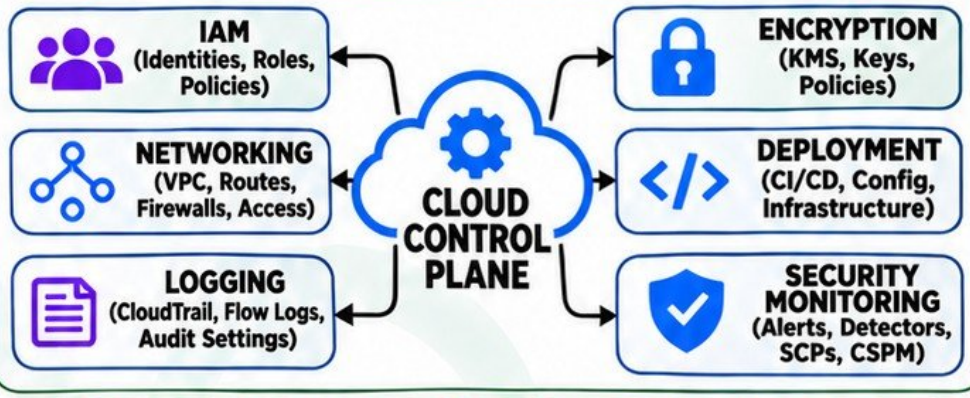
THE CLOUD CONTROL PLANE IS YOUR HIGHEST-VALUE TARGET

Why Control Over Configuration Can Be More Dangerous Than Data Access

THE IDEA

The cloud control plane lets you change how the environment works — who can access it, how it connects, what it logs, how it encrypts data, and what gets deployed.

CLOUD CONTROL PLANE AT A GLANCE



WHAT AN ATTACKER CAN DO

- Modify IAM (create users/roles, grant access).
- Change networking (open access, disable controls).
- Alter logging (disable or reduce audit visibility).
- Modify encryption (change KMS keys or policies).
- Tamper with deployments (approve, replace, or push malicious code).
- Disable security monitoring (alerts, detections, CSPM, etc.).

REAL-WORLD IMPACT OF CONTROL PLANE ACCESS

CHANGE MADE BY ATTACKER	RESULT
Create a new admin role	Persistent privileged access.
Modify network rules	Expose private resources to the internet.
Disable or reduce logging	Hide their activity and make investigation hard.
Change KMS policy	Gain access to encrypted data later.
Alter CI/CD configuration	Deploy malicious or backdoored workloads.
Disable security alerts	Delay detection and extend access.

REVIEW AND DEFEND

1. Identify who can change IAM, network, logging, encryption and CI/CD.
2. Enforce least privilege and separation of duties.
3. Protect critical changes with approval and change controls.
4. Enable immutable, centralized logging and alerts.
5. Continuously monitor for configuration changes and policy drift.

SENIOR ENGINEER INSIGHT

An attacker who controls infrastructure configuration can weaken security without directly accessing application data at first.

VERIQTA

09

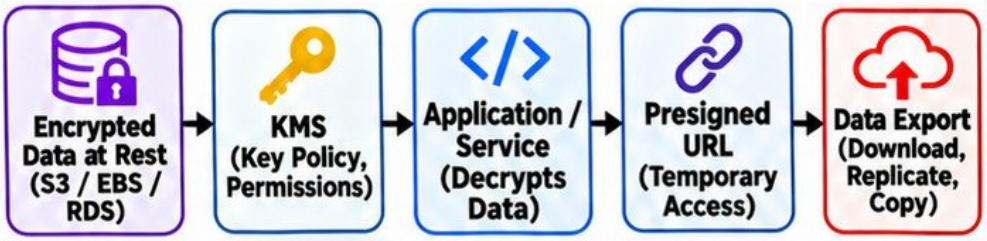
ENCRYPTED DATA, EXPOSED BUSINESS

How Encryption, Access, and Data Export Can Still Lead to a Breach

THE IDEA

Data can be encrypted at rest, yet still be read or exported when an application or service has the right permissions, keys, or data access paths.

DATA FLOW: FROM ENCRYPTED TO EXPOSED



KEY COMPONENTS

- **KMS Permissions** – who can use the key (e.g., kms:Decrypt).
- **Decrypt-Capable Services** – services that can read data using a KMS key (e.g., EC2, Lambda, RDS, S3).
- **Presigned URLs** – time-limited links that allow data access without re-authentication.
- **Application Authorization** – app-level controls that determine who can read, export, or share data.
- **Data Export Paths** – ways data can leave (e.g., download, cross-account copy, replication, backups).

COMMON EXPOSURE SCENARIOS

SCENARIO	WHAT HAPPENS
Over-permissive KMS	A compromised role can decrypt and read encrypted data.
Trusted service misuse	A service (e.g., EC2, Lambda) with decrypt access leaks data.
Presigned URL abuse	A valid link is shared, stolen, or used before it expires.
Application logic gap	The app allows unauthorized users to export or bulk-download data.
Data replication/export	Data is copied to another account, region, or external storage.

REVIEW AND HARDEN

- Apply least privilege on KMS keys and key policies.
- Audit which services and roles can use kms:Decrypt.
- Limit presigned URLs (short TTL, minimal scope, IP rules).
- Enforce strong application authorization and export controls.
- Monitor, log, and alert on decryption and large data export activity.

SENIOR ENGINEER INSIGHT

Encryption at rest does not prevent an authorized but compromised application identity from reading or exporting plaintext.

VERIQTA

10

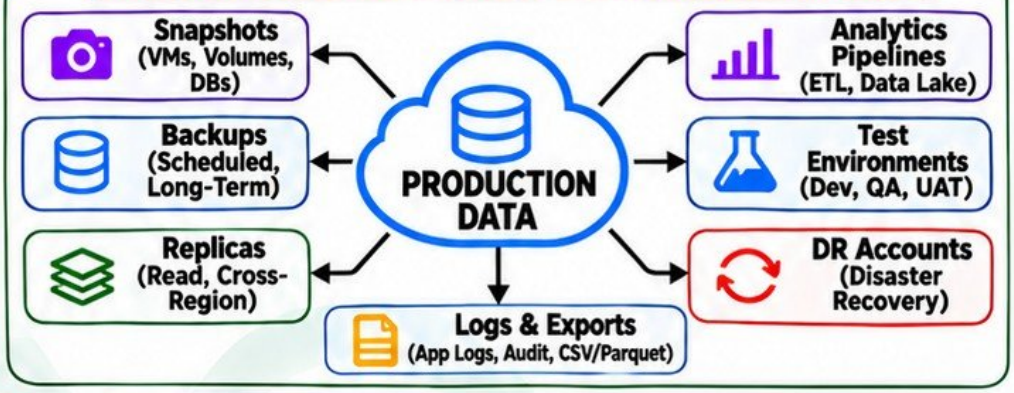
THE FORGOTTEN COPIES OF PRODUCTION DATA

How Data Spreads, Persists, and Becomes a Hidden Risk

THE IDEA

Production data is often copied to many places for backup, analysis, testing, and recovery. Each copy creates another potential access path, and weaker controls in any environment can expose sensitive data.

THE DATA LIFECYCLE: MULTIPLE COPIES, MORE RISK



COMMON DATA COPIES

- **Snapshots** – point-in-time copies of compute or database data.
- **Backups** – periodic copies for retention and restore.
- **Logs** – application, access, and audit data that may contain sensitive information.
- **Analytics Pipelines** – data copied to warehouses or lakes (e.g., S3, Redshift, BigQuery).
- **Test Environments** – production-like data used for development and QA.
- **Replicas** – read replicas or cross-region copies for availability.
- **Exports** – data extracts (e.g., CSV, Parquet) for reporting or integration.
- **DR Accounts** – full or partial copies in a recovery environment.

RISK EXAMPLES

DATA COPY	REAL RISK
Snapshots	Older snapshots still contain sensitive data.
Backups	Long-retained backups with weaker access controls.
Logs	Sensitive data (e.g., tokens, PII) in logs.
Analytics	Broad access to data lakes or reporting tools.
Test Environments	Production data in non-production with loose controls.
Replicas	Cross-region or read replicas accessible by more users.
Exports	Data files shared externally or stored in unmanaged locations.
DR Accounts	Recovery environments with outdated or permissive access.

REVIEW CHECKLIST

- Inventory all copies (snapshots, backups, logs, replicas, exports, DR).
- Verify access controls and encryption on every copy.
- Purge or anonymize old and unnecessary copies.
- Ensure test and analytics environments do not use real production data unless strictly required and protected.
- Include copies in your regular access reviews and risk assessments.

SENIOR ENGINEER INSIGHT

Production access controls are incomplete if older copies of the same data remain accessible through weaker environments.

VERIQTA

11

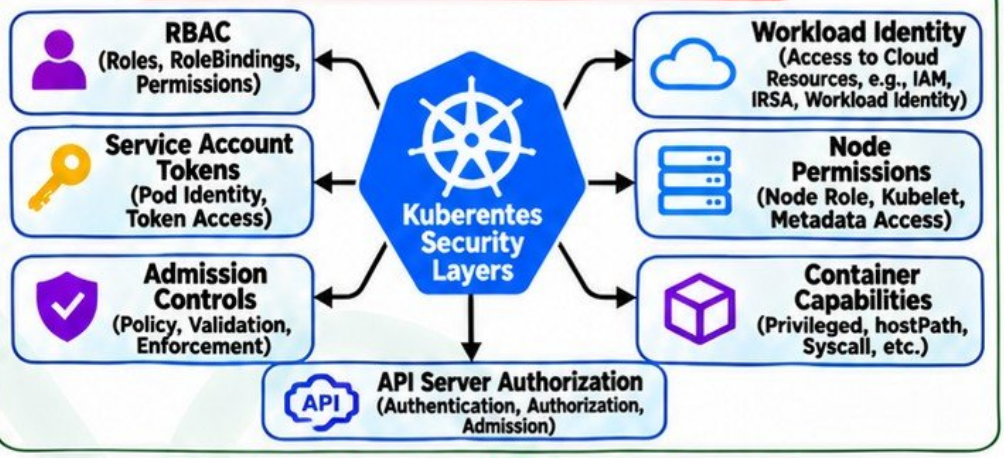
KUBERNETES RBAC IS NOT THE WHOLE SECURITY BOUNDARY

Why Cluster Permissions Alone Do Not Guarantee Real Security

THE IDEA

Kubernetes RBAC controls what users, service accounts, and applications can do inside the cluster, but it is only one layer. A workload can still gain high-privilege access through tokens, node access, cloud identity, or container settings.

THE BIGGER SECURITY PICTURE



KEY COMPONENTS

- **RBAC** – defines what a user or service account can do.
- **Service Account Tokens** – credentials used by pods to access the API server.
- **Admission Controls** – policies (e.g., OPA, Kyverno) that enforce security rules.
- **Workload Identity** – links workloads to cloud IAM (e.g., IRSA, GKE WI).
- **Node Permissions** – access to the node (e.g., metadata, host resources).
- **Container Capabilities** – extra privileges (e.g., privileged, hostPath, NET_ADMIN).

REAL-WORLD RISK SCENARIOS

SCENARIO	HOW ACCESS CAN BE GAINED
Pod with limited RBAC	Uses node or cloud credentials to gain higher access.
Service account token leak	Token used to call the API server directly (even from outside the pod).
Weak admission controls	A privileged or overly permissive pod is deployed.
Workload identity misuse	Pod assumes a cloud IAM role with broad permissions.
Node compromise	Attacker accesses node metadata or other pods.
Over-permissive capabilities	Container escapes limits and gains host or cluster access.

REVIEW AND HARDEN

1. Apply least-privilege RBAC for users and service accounts.
2. Restrict service account token use and set short token lifetimes.
3. Enforce strong admission controls (e.g., OPA, Kyverno, Pod Security).
4. Use workload identity with tightly scoped cloud IAM roles.
5. Harden nodes (deny metadata access, patch, and monitor).
6. Remove unnecessary container capabilities and avoid privileged pods.

SENIOR ENGINEER INSIGHT

Restrictive Kubernetes RBAC cannot compensate for a workload that can obtain powerful node or cloud credentials through another path.

VERIQTA

12

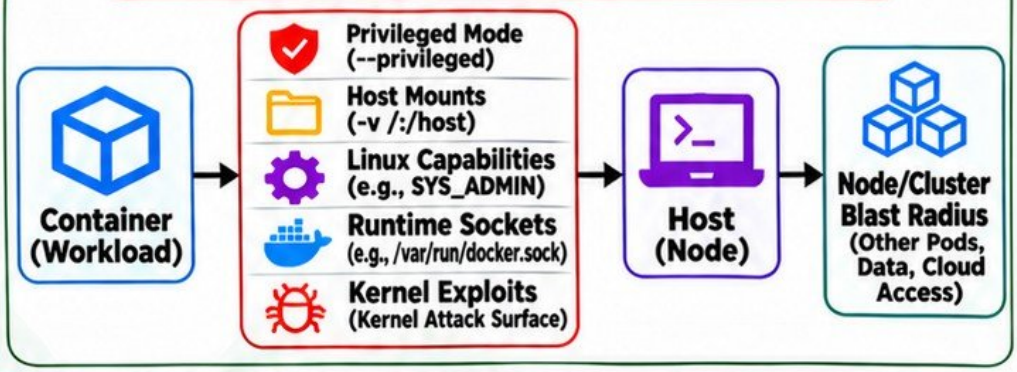
CONTAINER ISOLATION FAILS AT THE HOST BOUNDARY

Why Containers Can Still Lead to Host and Cluster Compromise

THE IDEA

Containers are isolated processes, not full virtual machines. When a container is given too much access or misconfigured, it can break out to the host and impact the entire node or cluster.

FROM CONTAINER TO HOST: ATTACK PATHS



KEY COMPONENTS

- **Privileged Containers** – run with almost full host access.
- **Host Mounts** – expose host file systems (e.g., -v /var/run/docker.sock).
- **Linux Capabilities** – fine-grained privileges (e.g., SYS_ADMIN, NET_ADMIN).
- **Runtime Sockets** – access to the container runtime (e.g., Docker, containerd) to control other containers.
- **Kernel Attack Surface** – exploiting kernel vulnerabilities to escape the container.
- **Node Blast Radius** – compromise of the host can impact all containers on the node and the cluster.

COMMON MISCONFIGURATIONS AND IMPACT

MISCONFIGURATION	WHAT IT ALLOWS	POTENTIAL IMPACT
Privileged container	Near full host access	Container escape, node compromise
Host mount (e.g., /:/host)	Read/write host file system	Access to host data, secrets, and configurations
Extra capabilities (e.g., SYS_ADMIN)	Perform sensitive host operations	Modify system settings, mount file systems, etc.
Runtime socket access (/var/run/docker.sock)	Control the container runtime	Start/stop containers, run privileged containers
Unpatched kernel	Exploit kernel vulnerabilities	Escalate to host and potentially the cluster

REVIEW AND HARDEN

1. Avoid --privileged unless absolutely necessary.
2. Do not mount sensitive host paths (e.g., /, /var/run/docker.sock).
3. Use least-privilege Linux capabilities (drop all, add only what is needed).
4. Keep the host kernel and container runtime updated.
5. Enforce Pod Security Standards/Policies (e.g., Baseline or Restricted).
6. Continuously audit container, node, and runtime configurations.

SENIOR ENGINEER INSIGHT

A container is not a virtual machine. Workload isolation depends on the combined strength of kernel boundaries, runtime configuration, and node permissions.

VERIQTA

13

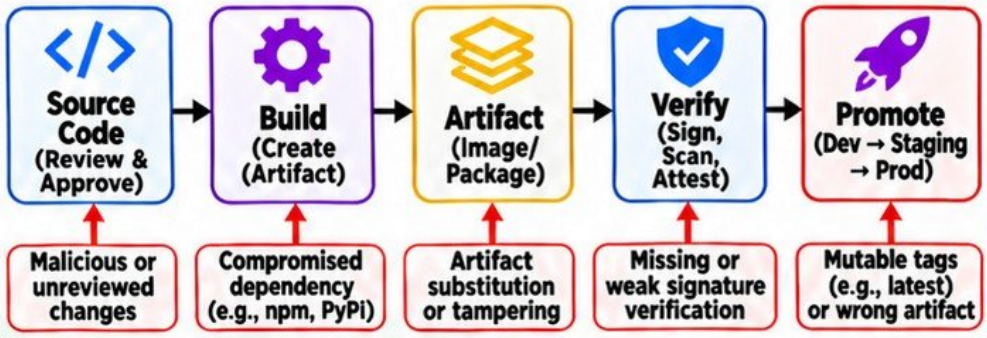
THE DEPLOYMENT WAS APPROVED. THE ARTIFACT WAS NOT.

Why a Trusted Pipeline Can Still Deliver an Untrusted Artifact

THE IDEA

A deployment can be properly approved, yet the actual artifact that reaches production may be different, tampered with, or built from compromised dependencies.

THE PIPELINE AND RISK POINTS



KEY COMPONENTS

- **Build Provenance** – verifiable record of how, when, and from what commit the artifact was built.
- **Artifact Substitution** – replacing a valid artifact with a malicious one after approval.
- **Dependency Compromise** – malicious or backdoored third-party packages (e.g., npm, PyPI, Maven).
- **Image Signatures** – cryptographic signatures to prove artifact integrity (e.g., Sigstore, Cosign).
- **Mutable Tags** – changing what a tag (e.g., latest) points to over time.
- **Environment Promotion** – moving an artifact across environments (dev, staging, prod) with proper controls.

COMMON RISKS AND EXAMPLES

RISK AREA	EXAMPLE	IMPACT
Build provenance	No proof of which commit produced the artifact.	Cannot trust what was deployed.
Artifact substitution	A different image is pushed after approval.	Malicious or untested code in production.
Dependency compromise	A third-party library is backdoored.	Attacker code runs with your application.
Image signatures	Artifact is not signed or signature not verified.	Tampered image can be deployed.
Mutable tags	"latest" is retagged to a new, untrusted image.	Production runs a different build than reviewed.
Environment promotion	No integrity check between environments.	A safe artifact in staging is replaced in production.

REVIEW AND HARDEN

1. Generate and verify build provenance (e.g., SLSA, SBOM, attestations).
2. Use signed artifacts and enforce signature verification (e.g., Cosign, Notary).
3. Pin and verify dependencies (lock files, checksums, trusted registries).
4. Avoid mutable tags; use immutable, versioned tags (e.g., v1.2.3, commit SHA).
5. Enforce promotion gates with integrity and policy checks between environments.
6. Continuously scan artifacts for vulnerabilities and supply-chain risks.

SENIOR ENGINEER INSIGHT

Approving a deployment pipeline is not proof that the exact artifact reaching production is the one that was reviewed and tested.

VERIQTA

14

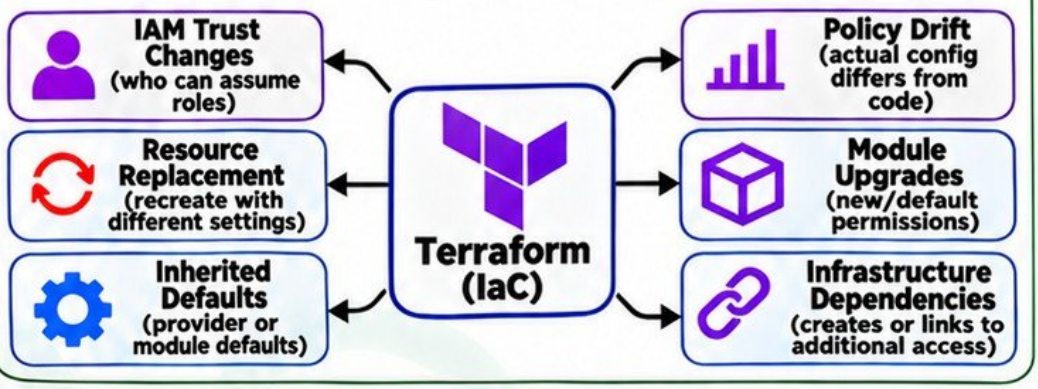
TERRAFORM PLANS THAT HIDE SECURITY CONSEQUENCES

Why a Small IaC Change Can Create a Big Security Risk

THE IDEA

A Terraform plan can look harmless, but changes to identity trust, resource behavior, defaults, or dependencies can quietly expand access or weaken your security posture.

COMMON TERRAFORM CHANGES WITH SECURITY IMPACT



KEY COMPONENTS

- **IAM Trust Changes** – modifying who can assume a role (e.g., adding a new principal).
- **Resource Replacement** – destroying and recreating a resource, which may change permissions, endpoints, or exposure.
- **Inherited Defaults** – provider or module defaults (e.g., public access, broad IAM policies).
- **Policy Drift** – the live infrastructure differs from the Terraform code.
- **Module Upgrades** – updating a module that introduces new resources or permissions.
- **Infrastructure Dependencies** – creating or linking resources (e.g., add a role, attach a policy, open a network path).

REAL-WORLD EXAMPLES

CHANGE IN PLAN	SECURITY CONSEQUENCE
Add a principal to an IAM role trust policy	A new user, service, or account can now assume the role.
Replace a load balancer or database	The new resource may be publicly accessible or use weaker settings.
Rely on a module default (e.g., public = true)	Creates unintended internet exposure (e.g., public S3, open security group).
Policy drift (manual change outside Terraform)	Actual permissions are broader than the code shows.
Upgrade a Terraform module (e.g., from v1.2 to v1.3)	The new version may create extra resources or grant wider access.
Add a dependency (e.g., attach a policy, VPC link)	Can introduce new access paths or cross-account trust.

REVIEW AND HARDEN

- Review IAM trust and permission changes in every plan.
- Check for resource replacements and their new configuration.
- Understand module upgrades and any new default behavior.
- Detect and fix policy drift (e.g., terraform plan -refresh-only).
- Assess downstream dependencies (roles, policies, networks, links).
- Use policy-as-code and guardrails (e.g., OPA, tfsec, Checkov) in CI/CD.

SENIOR ENGINEER INSIGHT

A small infrastructure change can alter a major trust boundary. Review the resulting permissions and exposure, not merely the number of changed resources.

VERIQTA

15

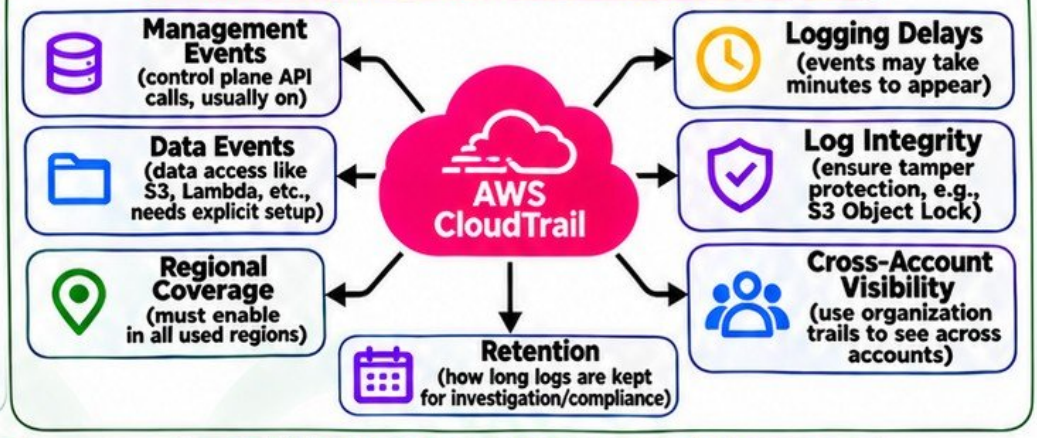
CLOUDTRAIL IS ENABLED. WHAT CAN YOU ACTUALLY PROVE?

Why Enabled Logging May Still Leave Gaps in Your Investigation

THE IDEA

CloudTrail records API activity in your AWS account, but out-of-the-box logging may not capture everything you need to prove what happened, who did it, and how the access was used.

CLOUDTRAIL COVERAGE AND KEY GAPS



KEY COMPONENTS

- **Management Events** – control plane actions (e.g., IAM changes, resource creation, configuration).
- **Data Events** – data plane activity (e.g., S3 object access, Lambda invocation) that must be enabled.
- **Regional Coverage** – CloudTrail is region-specific; enable in all relevant regions.
- **Event Selectors** – rules that define which data events to log (e.g., specific S3 buckets, Lambda).
- **Logging Delays** – events are not always real time (typically seconds to minutes).
- **Retention & Integrity** – set appropriate retention and protect logs from modification (e.g., S3 Object Lock).
- **Cross-Account Visibility** – use an organization trail to collect logs from multiple accounts.

COMMON GAPS AND REAL EXAMPLES

GAP	EXAMPLE	RISK
Missing data events	S3 object read not logged (no data event enabled)	Cannot prove who accessed the data.
Partial regional coverage	CloudTrail enabled in us-east-1 only, but activity in eu-west-1	Important events are not recorded.
Narrow event selectors	Logs only one S3 bucket, others not included	Gaps in data access activity.
Logging delays	Event appears several minutes later	Slower detection and response.
Short retention	Logs kept for 30 days only	Evidence may be lost before investigation.
No integrity protection	Logs stored in an editable S3 bucket	Logs could be deleted or altered.
Limited account visibility	Only single-account trail	Activity in other accounts remains unseen.

REVIEW AND HARDEN

1. Enable organization or multi-account CloudTrail.
2. Turn on required data events (e.g., S3, Lambda, DynamoDB).
3. Use event selectors to cover all critical resources.
4. Enable logging in every active region.
5. Protect log integrity (e.g., S3 Object Lock, immutable storage).
6. Set appropriate retention and monitor for logging failures.

SENIOR ENGINEER INSIGHT

Enabled logging is not the same as sufficient forensic evidence. Some actions require additional event sources or explicit data-event configuration.

VERIQTA

16

THE ATTACKER DISABLED YOUR EVIDENCE BEFORE YOUR ALERT

How Attackers Hide, Destroy, or Blind Your Security Visibility

THE IDEA

A skilled attacker can gain access, change or disable logging, remove or delay alerts, perform destructive actions, and target your security tools to hide their activity and reduce your chances of detection and investigation.

ANATOMY OF THE INCIDENT (EXAMPLE FLOW)

1. **Initial Access** (e.g., stolen credentials or IAM abuse)
2. **Logging Changes** (e.g., disable CloudTrail, edit event selectors, stop log exports)
3. **Detection Gaps** (e.g., disable alerts, modify detection rules, reduce visibility)
4. **Destructive Actions** (e.g., delete data, encrypt resources, remove snapshots/ backups)
5. **Security Tool Targeting** (e.g., tamper with SIEM, EDR, or monitoring agents)

KEY COMPONENTS

- **Logging Changes** – disabling, reducing, or modifying log collection (e.g., CloudTrail, S3, VPC).
- **Detection Gaps** – removing or muting alerts and detection rules (e.g., CloudWatch, SIEM).
- **Destructive Actions** – deleting data, snapshots, backups, or critical resources.
- **Security Tool Compromise** – weakening or removing security agents, log forwarders, or EDR.
- **Independent Logging** – sending logs to a separate, controlled account or external system.
- **Immutable Retention** – write-once, read-many (WORM) or lockable retention for logs.

INCIDENT TIMELINE (ILLUSTRATIVE EXAMPLE)

TIME	ATTACKER ACTION	IMPACT
00:00	Compromises an IAM user (with elevated permissions)	Gains control of cloud resources.
00:10	Disables or limits CloudTrail (e.g., remove S3 log delivery)	New actions are no longer logged centrally.
00:15	Disables alerting/detection (e.g., CloudWatch rules, SIEM)	Security team does not get notified.
00:25	Deletes or encrypts critical data (e.g., S3, EBS snapshots)	Data loss and service disruption.
00:30	Removes or tampers with security agents (e.g.,EDR, log agentes)	Reduces ongoing visibility and forensic evidence.

REVIEW AND HARDEN

1. Send logs to an independent account or external service.
2. Restrict who can change logging, alerts, and security tools (separation of duties).
3. Enable immutable log retention (e.g., S3 Object Lock / WORM).
4. Monitor for changes to logging, alerting, and security configurations.
5. Keep regular, tested backups and recovery procedures.
6. Use external or cross-account monitoring for tamper detection.

SENIOR ENGINEER INSIGHT

Independent logging destinations, restricted administration, immutable retention controls, and external monitoring improve your ability to investigate a compromised account.

VERIQTA

17

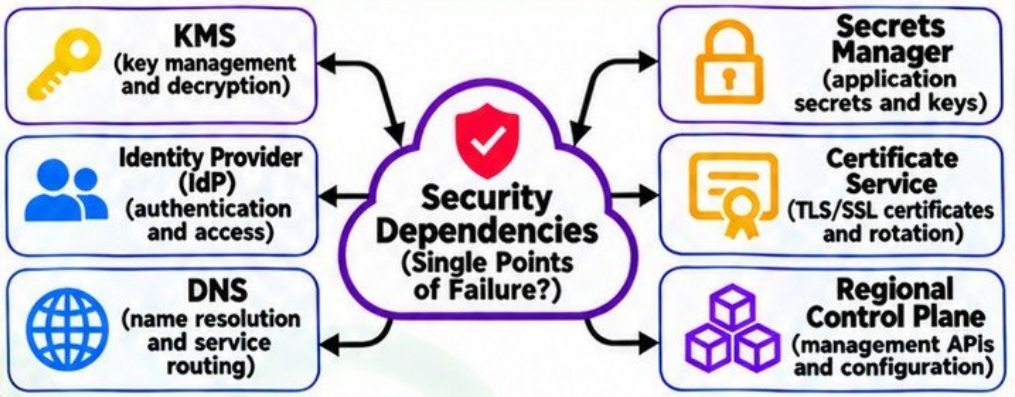
A SECURE ARCHITECTURE THAT CANNOT SURVIVE AN OUTAGE

Why Security Controls Must Also Be Designed for Resilience

THE IDEA

Your architecture may be secure in normal operation, but if a key security service goes down, such as KMS, IdP, DNS, or a secrets manager, your applications, access and security controls can fail, causing service outage.

KEY SECURITY DEPENDENCIES AND OUTAGE IMPACT



KEY COMPONENTS

- **KMS** – used to encrypt, decrypt, and manage encryption keys.
- **Identity Provider (IdP)** – handles authentication and access (e.g., SSO, OAuth, OIDC).
- **DNS** – resolves service names and enables application communication.
- **Secrets Manager** – stores and retrieves sensitive secrets (e.g., API keys, database credentials).
- **Certificate Service** – issues and rotates TLS/SSL certificates for secure communication.
- **Regional Control Plane** – cloud or platform management layer for configuration, orchestration, and policy enforcement.

OUTAGE SCENARIOS AND IMPACT

DEPENDENCY	OUTAGE EXAMPLE	POTENTIAL IMPACT
KMS	KMS API unavailable	Apps cannot decrypt data or start (e.g., encrypted volumes).
Identity Provider	IdP/SSO service down	Users and services cannot authenticate or get new tokens.
DNS	DNS outage or misconfiguration	Services become unreachable even if they are healthy.
Secrets Manager	Secrets service unavailable	Apps cannot retrieve credentials and may fail to start or scale.
Certificate Service	Certificate issuance/renewal fails	TLS certificates expire, breaking secure access (HTTPS, APIs).
Regional Control Plane	Regional control plane degraded	Cannot make changes, scale, or manage security policies.

REVIEW AND HARDEN

1. Identify all security and platform dependencies (KMS, IdP, DNS, etc.).
2. Design for high availability and regional redundancy.
3. Use caching, token lifetimes, and graceful degradation where safe.
4. Have break-glass access with strict controls and auditing.
5. Test outage scenarios (e.g., KMS or IdP unavailable).
6. Document and rehearse a resilience and recovery plan.

SENIOR ENGINEER INSIGHT

A security control can become a critical availability dependency. Resilience planning must preserve security boundaries during degraded operation.

VERIQTA

18

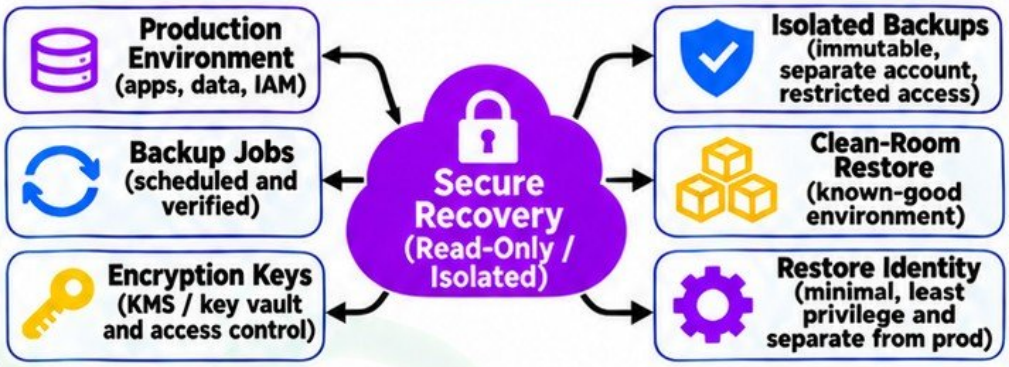
THE BACKUP PASSED. THE RECOVERY IS COMPROMISED.

Why Backups Alone Do Not Guarantee a Safe Recovery

THE IDEA

Backups can run successfully, yet recovery may still fail or be unsafe if an attacker has compromised your production account, admin credentials, keys, or the systems that control restoration.

RANSOMWARE-RESISTANT RECOVERY MODEL



KEY COMPONENTS

- **Ransomware-Resistant Recovery** – the ability to restore data and services even after a compromise (e.g., immutable backups, separate account).
- **Backup Isolation** – keeping backups in a separate account or network with restricted access.
- **Restore Identities** – dedicated, least-privilege roles used only for recovery.
- **Key Dependencies** – access to the encryption keys (e.g., KMS) needed to decrypt and restore data.
- **Immutable Copies** – write-once, read-many (WORM) backups that cannot be altered or deleted.
- **Clean-Room Restoration** – restoring in a new, isolated environment to verify integrity before going live.

COMMON FAILURE SCENARIOS AND IMPACT

SCENARIO	WHAT HAPPENS	IMPACT
Attacker gains admin access	Modifies, deletes, or limits backups	No valid restore point available.
KMS or key access compromised	Attacker disables or denies key use	Backups exist but cannot be decrypted.
Backups in the same account	Attacker changes policies or deletes backups	Backups are lost or made unusable.
No immutable copies	Backups can be altered encrypted or deleted	Recovery produces tampered or empty data.
No clean-room restore	Restores directly into a compromised environment	The attacker regains access after recovery.

REVIEW AND HARDEN

1. Keep backups in a separate account with strict access controls.
2. Use immutable (WORM) backup storage and retention policies.
3. Protect and test access to encryption keys (e.g., KMS).
4. Use dedicated, least-privilege restore identities.
5. Perform regular, end-to-end recovery tests (including clean-room).
6. Document and validate a ransomware recovery runbook.

SENIOR ENGINEER INSIGHT

A successful backup job does not prove that recovery remains possible after an attacker compromises the production account and its administrative credentials.

VERIQTA

19

THE SECURITY FINDING THAT SHOULD NOT BE FIXED FIRST

Why Risk-Based Prioritization Beats Severity Alone

THE IDEA

Not all high-severity findings should be fixed first. Prioritize based on real risk—how easily it can be exploited, what access it provides, what assets it reaches, and the potential business impact.

RISK-BASED PRIORITIZATION FRAMEWORK



KEY COMPONENTS

- ✓ **Exploitability** – the likelihood and effort required to exploit.
- ✓ **Effective Privilege** – the level of access gained (e.g., read, write, admin).
- ✓ **Reachable Assets** – criticality and sensitivity of the systems or data that can be reached.
- ✓ **Blast Radius** – the potential scope of impact (e.g., one service vs. entire production).
- ✓ **Compensating Controls** – existing measures that reduce or contain the risk (e.g., network isolation, WAF, IAM guardrails).
- ✓ **Business Consequences** – the real-world impact (e.g., downtime, data loss, compliance, or customer trust).

EXAMPLE: WHICH FINDING TO FIX FIRST?

FINDING	SEVERITY	RISK CONTEXT	PRIORITY
RCE on a non-production isolated service	Critical	• Isolated, no production data • Not internet-exposed • Low blast radius	Lower (Fix after higher risk)
IAM role misconfiguration (with broad access)	Medium	• Can access production resources • Reachable from multiple services • High blast radius (data, infra, cost, and integrity)	Higher (Fix first)

REVIEW AND HARDEN

1. Go beyond severity and assess real-world risk.
2. Identify the actual attack path and reachable assets.
3. Evaluate the effective privilege and blast radius.
4. Account for compensating controls that reduce risk.
5. Prioritize fixes that protect production trust boundaries.
6. Document the risk rationale and fix order.

SENIOR ENGINEER INSIGHT

A critical-severity vulnerability on an isolated workload may present less immediate risk than a lower-rated identity misconfiguration exposing a production trust boundary.

VERIQTA

20

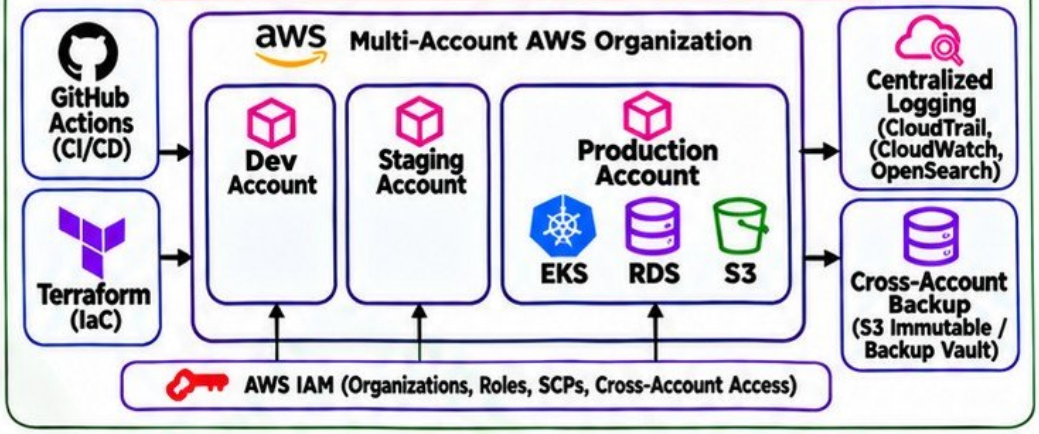
FINAL CHALLENGE: APPROVE OR REJECT THIS CLOUD ARCHITECTURE

A Production Security Review of a Multi-Account AWS Platform

THE CHALLENGE

You are the Principal Engineer (Security Reviewer). Review a multi-account AWS platform running EKS, RDS, S3, GitHub Actions, Terraform, centralized logging, and cross-account backup, then decide: Approve or Reject for production.

ARCHITECTURE OVERVIEW (SIMPLIFIED)



YOUR TASKS

- ✓ **Identify attack paths** (e.g., CI/CD, IAM, EKS, S3, RDS).
- ✓ **Establish evidence** (from design, policies, configs, and logs/diagrams).
- ✓ **Rank risks** (Critical, High, Medium, Low) with clear reasoning.
- ✓ **Recommend mitigations** (practical and prioritized).
- ✓ **Document accepted residual risk** (what remains and why it is acceptable).
- ✓ **Deliver a defensible decision** (Approve or Reject) with conditions, if any.

EXAMPLE REVIEW OUTPUT (ILLUSTRATIVE)

AREA	KEY RISK / ATTACK PATH	RISK	RECOMMENDED MITIGATION
IAM	Over-permissive roles or cross-account trust abuse.	Critical	Enforce least privilege, add permission boundaries, review SCPs.
CI/CD	Pipeline token compromise (GitHub Actions) can deploy malicious changes.	High	Use OIDC with short-lived roles, repo branch protection, signed artifacts.
EKS	Cluster privilege escalation (e.g., overly broad RBAC or node access).	High	Harden RBAC, restrict node IAM, remove privileged containers.
S3	Data exposure or deletion (e.g., public access, weak policies).	Medium	Block public access, enforce KMS encryption, versioning, and access logs.
RDS	Unauthorized access or data exfiltration.	Medium	Private subnets, strict security groups, IAM auth, encryption at rest and in transit.
Logging	Incomplete visibility or log tampering.	Medium	Centralized, immutable logs (S3 Object Lock), cross-account, and alerting.
Backup	Backups not recoverable or deletable by an attacker.	Medium	Cross-account backup vault, immutability, access separation, and restore testing.

REVIEW CHECKLIST (DECISION INPUT)

- ☐ Threat model and attack paths reviewed.
- ☐ IAM, RBAC, and cross-account access validated.
- ☐ Encryption (KMS) and key access reviewed.
- ☐ Logging, alerting, and tamper resistance verified.
- ☐ Backup and restore tested (incl. cross-account).
- ☐ Dependencies and failure impact assessed.
- ☐ Residual risks documented and owned.
- ☐ Clear decision: Approve, Reject, or Approve with conditions.

SENIOR ENGINEER INSIGHT

A Principal Engineer does not approve an architecture because every checklist item is green. Approval depends on whether critical risks are understood, bounded, evidenced, and owned.